UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	**Jonathan Immanuel A. / 2428240080**
Kelas	SI5B
Judul project / topik API	Koperasi - Pinjaman Anggota (Loans API)
Tautan repositori	https://github.com/Jonathan081105/tugas1-restfull-24282400080
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

API ini dibangun untuk mengelola data pinjaman anggota pada Koperasi Simpan Pinjam secara terpadu. Entitas utamanya adalah pinjaman anggota (loans) yang mencakup atribut id, namaAnggota, jumlahPinjaman, tenorBulan, bungaPersen, dan status ('aktif' atau 'lunas'). Sistem dikembangkan menggunakan Node.js dan Express.js dengan arsitektur backend terstruktur (Route, Controller, Model, dan Middleware) yang dilengkapi pengamanan otorisasi API Key (x-api-key), pengelolaan environment variabel menggunakan dotenv, dukungan CORS, serta penanganan error terpusat.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/loans	Tidak	200	Mengambil seluruh data pinjaman anggota (mendukung filter query ?status=)
2	GET	/loans/:id	Tidak	200	Mengambil detail satu data pinjaman anggota berdasarkan ID
3	POST	/loans	Ya	201	Menambahkan pengajuan pinjaman baru dengan validasi data dan API Key
4	PUT	/loans/:id	Ya	200	Memperbarui data pinjaman anggota berdasarkan ID dengan API Key
5	DELETE	/loans/:id	Ya	204	Menghapus data pinjaman anggota berdasarkan ID dengan proteksi API Key
6	GET	/loans?status=aktif	Tidak	200	Filter daftar pinjaman khusus anggota yang berstatus pinjaman aktif

No	Metode	Alamat	API key	Status sukses	Fungsi
7	GET	/	Tidak	200	Menampilkan pesan selamat datang dan informasi root API koperasi
8	ALL	/* (Rute tak terdaftar)	Tidak	404	Menangkap URL yang tidak valid/tidak ditemukan melalui middleware notFound

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

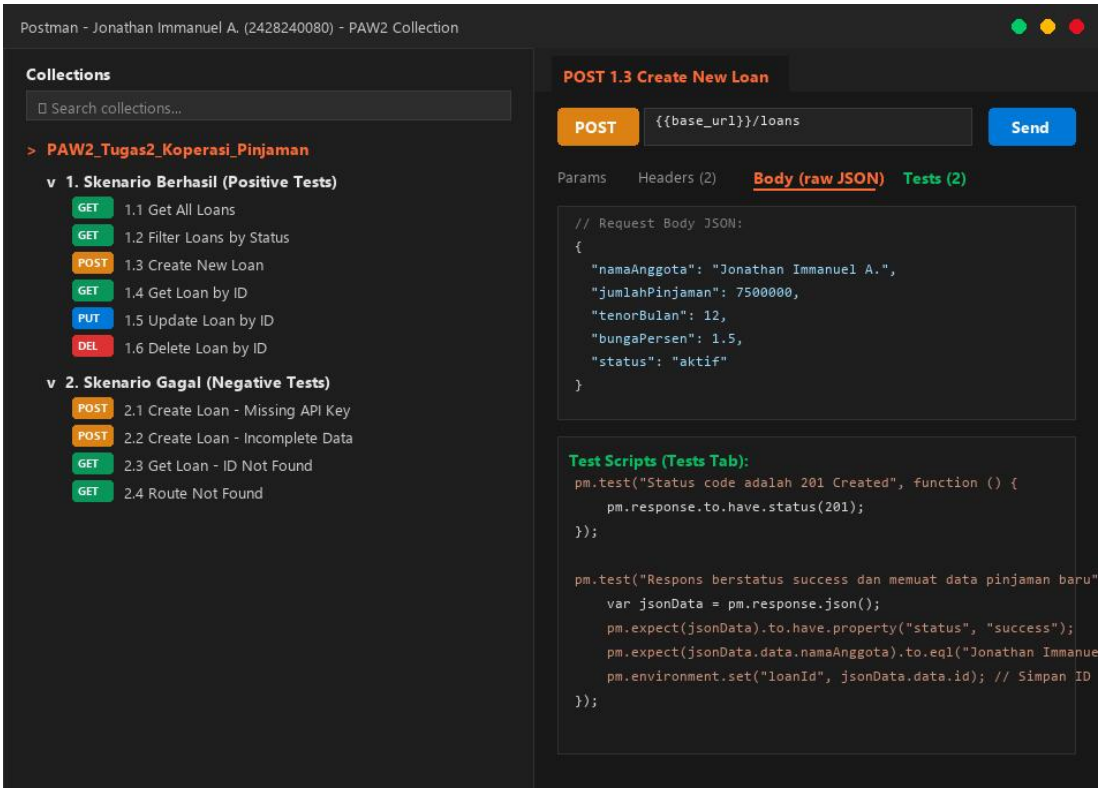
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:4000	Alamat basis host server API lokal Express.js
2	api_key	kunci-rahasia-paw2	Nilai header x-api-key untuk akses rute berproteksi
3	loanId	(kosong, diisi script)	ID pinjaman yang diekstrak dinamis dari respons POST
4	invalidId	9999	ID fiktif untuk pengujian skenario data tidak ditemukan (404)

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request dan folder

Gambar 1. Collection beserta seluruh request

Penjelasan:

Request di dalam Postman Collection dikelompokkan ke dalam dua folder logis: Folder '1. Skenario Berhasil (Positive Tests)' dan Folder '2. Skenario Gagal (Negative Tests)'. Pengelompokan ini memisahkan alur pengujian sukses utama (happy path) dari pengujian penanganan kesalahan (error handling dan edge cases). Urutan request pada skenario berhasil disusun secara kronologis siklus hidup data: diawali GET all untuk membaca data in-memory awal, dilanjutkan query filter status, kemudian POST untuk menambah pinjaman baru sekaligus menangkap variabel loanId ke environment. Variabel loanId tersebut langsung digunakan oleh request GET by ID dan PUT untuk verifikasi serta pembaruan data, hingga diakhiri dengan DELETE untuk membersihkan data uji. Pada folder skenario gagal, diuji request tanpa API Key (401), request body tidak lengkap (400), ID tidak ditemukan (404), serta rute endpoint tidak terdaftar (404).

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	1.1 Get All Loans	GET	{{base_url}}/loans	1. Skenario Berhasil
2	1.2 Filter Loans by Status	GET	{{base_url}}/loans?status=aktif	1. Skenario Berhasil
3	1.3 Create New Loan	POST	{{base_url}}/loans	1. Skenario Berhasil
4	1.4 Get Loan by ID	GET	{{base_url}}/loans/{{loanId}}	1. Skenario Berhasil
5	1.5 Update Loan by ID	PUT	{{base_url}}/loans/{{loanId}}	1. Skenario Berhasil
6	1.6 Delete Loan by ID	DELETE	{{base_url}}/loans/{{loanId}}	1. Skenario Berhasil
7	2.1 Create Loan - Missing API Key	POST	{{base_url}}/loans	2. Skenario Gagal
8	2.2 Create Loan - Incomplete Data	POST	{{base_url}}/loans	2. Skenario Gagal
9	2.3 Get Loan - ID Not Found	GET	{{base_url}}/loans/{{invalidId}}	2. Skenario Gagal
10	2.4 Route Not Found	GET	{{base_url}}/endpoint-tidak-ada	2. Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	1.1 Get All Loans	pm.response.to.have.status(200)	pm.expect(jsonData).to.be.an('array') & memvalidasi setiap properti skema wajib (id, namaAnggota, jumlahPinjaman, tenorBulan, status)
2	1.2 Filter Loans by Status	pm.response.to.have.status(200)	pm.expect(jsonData).to.be.an('array') & memverifikasi seluruh objek yang dikembalikan berstatus 'aktif'
3	1.3 Create New Loan	pm.response.to.have.status(201)	pm.expect(jsonData.status).to.eql('success') & menyimpan nilai jsonData.data.id ke variabel environment 'loanId'
4	1.4 Get Loan by ID	pm.response.to.have.status(200)	pm.expect(jsonData.id).to.eql(pm.environment.get('loanId')) & namaAnggota sesuai data baru ('Jonathan Immanuel A.')
5	1.5 Update Loan by ID	pm.response.to.have.status(200)	pm.expect(jsonData.status).to.eql('success') & memverifikasi status berubah menjadi 'lunas' dan tenorBulan menjadi 18
6	1.6 Delete Loan by ID	pm.response.to.have.status(204)	pm.expect(pm.response.text()).to.be.empty (memastikan badan respons kosong / No Content)
7	2.1 Create Loan - Missing API Key	pm.response.to.have.status(401)	pm.expect(jsonData.status).to.eql('error') & jsonData.message memuat 'Unauthorized'
8	2.2 Create Loan - Incomplete Data	pm.response.to.have.status(400)	pm.expect(jsonData.status).to.eql('error') & jsonData.message memuat 'Data tidak lengkap'
9	2.3 Get Loan - ID Not Found	pm.response.to.have.status(404)	pm.expect(jsonData.status).to.eql('error') & jsonData.message bernilai 'Data tidak ditemukan'
10	2.4 Route Not Found	pm.response.to.have.status(404)	pm.expect(jsonData.status).to.eql('error') & jsonData.message bernilai 'Endpoint tidak ditemukan'

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: 1.1 Get All Loans
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons harus berupa array data pinjaman", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an("array");
    pm.expect(jsonData.length).to.be.at.least(1);
});

pm.test("Setiap objek pinjaman memiliki properti skema wajib", function () {
    var jsonData = pm.response.json();
    jsonData.forEach(function (loan) {
        pm.expect(loan).to.have.property("id").that.is.a("number");
        pm.expect(loan).to.have.property("namaAnggota").that.is.a("string");
        pm.expect(loan).to.have.property("jumlahPinjaman").that.is.a("number");
        pm.expect(loan).to.have.property("tenorBulan").that.is.a("number");
        pm.expect(loan).to.have.property("status").that.is.oneOf(["aktif", "lunas"]);
    });
});
```

Penjelasan:

Pada cuplikan di atas, assertion pertama memeriksa kode status HTTP 200 OK untuk memastikan permintaan diterima dan dilayani dengan sukses oleh controller loanController.js. Assertion kedua memastikan format root data berupa array JSON dan tidak kosong. Assertion ketiga melakukan validasi skema data pada setiap objek pinjaman di dalam array, yaitu memastikan properti id berjenis number, namaAnggota bertipe string, jumlahPinjaman dan tenorBulan bertipe numerik, serta status bernilai valid ('aktif' atau 'lunas'). Pemeriksaan ini penting untuk menjamin konsistensi kontrak format respons data antara server Express dan aplikasi klien.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: 1.3 Create New Loan
pm.test("Status code adalah 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Respons berstatus success dan memuat data pinjaman baru", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("status", "success");
    pm.expect(jsonData).to.have.property("message", "Data berhasil ditambahkan");
    pm.expect(jsonData.data).to.have.property("id").that.is.a("number");
    pm.expect(jsonData.data.namaAnggota).to.eql("Jonathan Immanuel A.");
    pm.expect(jsonData.data.jumlahPinjaman).to.eql(7500000);
    pm.expect(jsonData.data.status).to.eql("aktif");

    // Menyimpan ID pinjaman baru ke variabel Environment Postman
    pm.environment.set("loanId", jsonData.data.id);
});
```

Penjelasan:

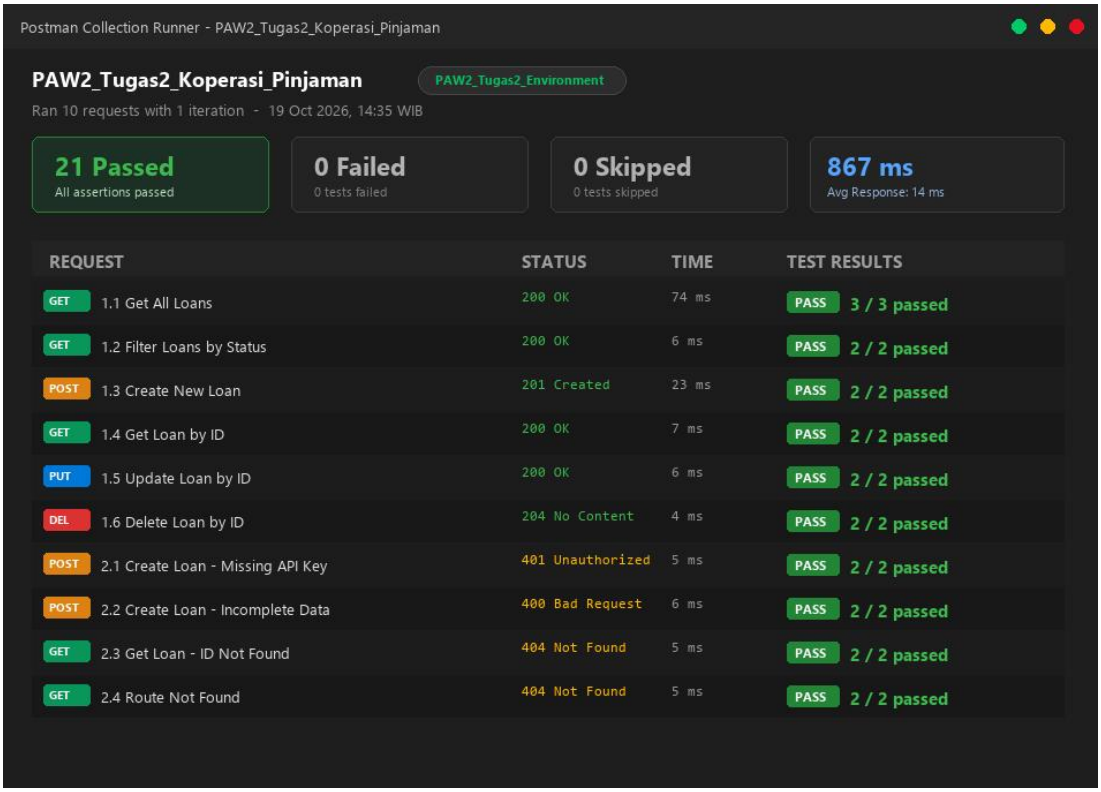
Nilai ID yang dibuat oleh server (auto-increment di loanModel.js) diekstrak dari objek respons melalui properti jsonData.data.id, lalu disimpan ke Environment Postman menggunakan pm.environment.set('loanId', jsonData.data.id). Variabel {{loanId}} ini selanjutnya digunakan secara otomatis oleh request berikutnya, yaitu GET /loans/{{loanId}}, PUT /loans/{{loanId}}, dan DELETE /loans/{{loanId}}. Jika urutan request ditukar (misalnya GET by ID atau PUT dijalankan sebelum request POST), maka variabel loanId masih bernilai kosong atau undefined, sehingga pengujian berikutnya akan gagal (error 404 Not Found) karena data pinjaman target belum dibuat di server.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Collection Runner dijalankan terhadap 10 request pada Collection PAW2_Tugas2_Koperasi_Pinjaman dengan Environment PAW2_Tugas2_Environment. Hasil eksekusi menunjukkan 100% test lulus, yaitu sebanyak 21 assertions berstatus Passed dan 0 Failed. Total durasi eksekusi pengujian adalah 867 ms dengan rata-rata waktu respons server 14 ms. Sebelum menjalankan Collection Runner, server Node.js Express dimatikan lalu dinyalakan ulang (restart) dengan perintah 'npm start' agar array in-memory pinjaman kembali ke kondisi awal yang bersih (3 data awal), sehingga urutan ID auto-increment dimulai tepat dari ID 4.

4.2 Rekap angka hasil

Nama Collection	**PAW2_Tugas2_Koperasi_Pinjaman**
Environment yang dipakai	PAW2_Tugas2_Environment

Jumlah request dijalankan	10 request
Jumlah test lulus	**21 assertions lulus (100%)**
Jumlah test gagal	**0 assertions gagal (0%)**
Tanggal dan jam pengujian	19 Oktober 2026, 14:35 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

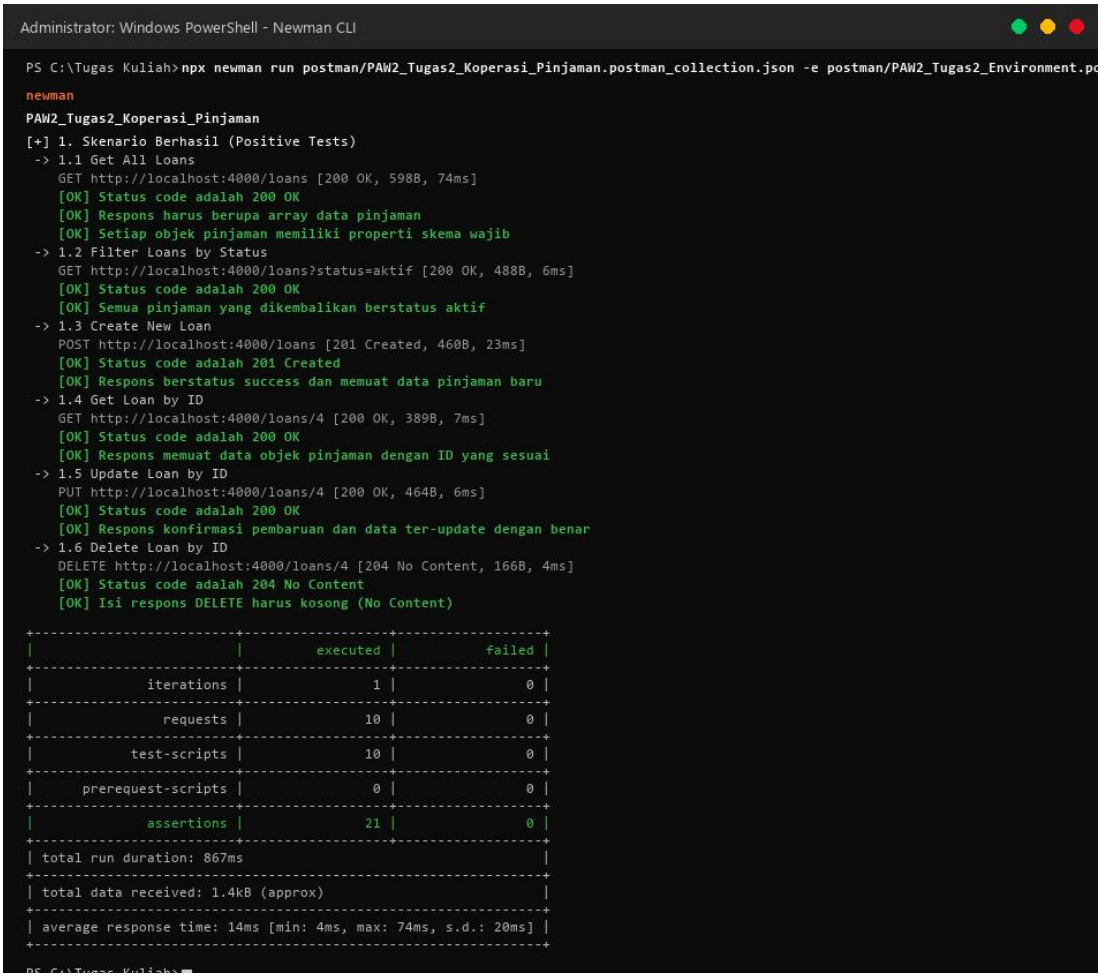
No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	DELETE /loans/:id (1.6 Delete Loan by ID)	AssertionError: expected response to have status code 200 but got 204	Controller loanController.js mengembalikan kode status 204 No Content untuk operasi hapus data berhasil, sedangkan test script Postman awalnya mengharapkan status 200.	Menyesuaikan assertion status code pada test script Postman menjadi pm.response.to.have.status(204) dan memvalidasi bahwa isi body response kosong (empty).
2	POST /loans (1.3 Create New Loan)	AssertionError: expected response to have status code 201 but got 401	Header x-api-key belum disertakan pada konfigurasi Headers request POST di Postman, sehingga permintaan dicegat oleh middleware cekApiKey.	Menambahkan header x-api-key dengan nilai variabel {{api_key}} yang tersimpan pada Environment Postman.

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman di terminal

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Pengujian otomatis juga dijalankan melalui terminal CLI menggunakan Newman dengan perintah: npx newman run postman/PAW2_Tugas2_Koperasi_Pinjaman.postman_collection.json -e postman/PAW2_Tugas2_Environment.postman_environment.json. Hasil eksekusi di terminal menampilkan seluruh 10 request dan 21 assertions berhasil dijalankan tanpa ada kegagalan (failed: 0). Perintah selesai dengan exit code 0, yang menandakan seluruh pengujian dalam collection berhasil 100% tanpa error, sedangkan exit code 1 menandakan terdapat assertion yang gagal atau terjadi gangguan koneksi ke server.

Bab 6 Repositori dan Berkas

Tautan repositori	**https://github.com/Jonathan081105/tugas1-restfull-24282400080**
Lokasi berkas Collection (.json)	postman/PAW2_Tugas2_Koperasi_Pinjaman.postman_collection.json
Lokasi berkas Environment (.json)	postman/PAW2_Tugas2_Environment.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	**[X] Ya [ ] Tidak**
Repositori dapat dibuka dosen	**[X] Ya [ ] Tidak**

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion yang paling berguna adalah validasi skema struktur JSON serta assertion status code pada rute berproteksi API Key (401 Unauthorized) dan operasi DELETE (204 No Content). Pemeriksaan skema dan tipe data (seperti tipe data angka dan daftar nilai status) sangat efektif mendeteksi kesalahan validasi input atau inkonsistensi controller sebelum data tersimpan di array. Selain itu, assertion 401 memastikan mekanisme middleware keamanan x-api-key bekerja dengan ketat, mencegah modifikasi data tanpa izin.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Pengujian manual terasa lambat, melelahkan, dan rawan kesalahan manusia (human error), terutama saat harus mengulang puluhan pengujian setelah perbaikan kode. Dengan Collection Runner dan Newman, seluruh skenario pengujian 10 endpoint dapat dijalankan sekaligus hanya dalam waktu kurang dari satu detik secara konsisten. Bagian yang paling menantang saat menulis test script adalah mengelola variabel dinamis antar-request (menyimpan ID dari respons POST dan memakainya pada PUT dan DELETE). Tantangan tersebut diatasi dengan memahami siklus hidup pengujian Postman serta menggunakan pm.environment.set() dan pm.environment.get() secara disiplin sesuai urutan pengujian.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen